AI-NATIVE VIDEO CREATION & EDITING PLATFORM

API Contract & Architecture Design

Version 1.0 • Technical Design Document • 04 November 2026

Purpose. Define how the React client, Express API, MongoDB, S3, Redis/BullMQ workers, FFmpeg renderer, and Gemini/OpenAI planning providers exchange data reliably and securely.

Companion documents: AI Video Editing Platform Functional & Technical Specification (v0.1) and AI Video Editing Platform Database Schema Blueprint (v0.1). This document extends their architecture with concrete request/response contracts and an explicit worker callback contract.

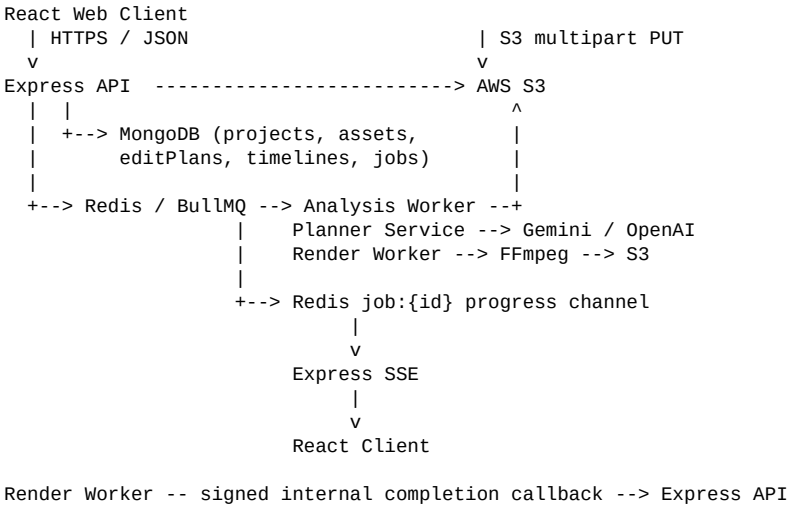
Document status and design assumptions

The existing project blueprint specifies React, Node.js/Express, MongoDB, FFmpeg, BullMQ/Redis, AWS S3, and Gemini/OpenAI. It defines presigned multipart uploads, prompt-to-Edit-Plan validation, preview/export queues, and browser progress through Server-Sent Events (SSE). The worker-to-API webhook below is a proposed internal contract to formalize completion notifications; the browser should continue to receive progress through SSE.

All endpoints below use the /api prefix. JSON examples are illustrative contracts; implementation may add fields while preserving backward compatibility. Authentication is assumed to use a verified bearer access token or secure session.

1. System architecture overview

The API is the control plane: it authenticates users, validates requests, creates database records, issues short-lived S3 URLs, and enqueues jobs. Large media bytes travel directly between the browser and S3 rather than through Express. Workers perform CPU-heavy analysis and rendering outside the request-response cycle.



1.1 Component responsibilities

Component	Responsibility / boundary
React client	Project UI, chunked upload orchestration, prompt entry, timeline state, preview playback, SSE subscription.
Express API	Authentication, tenant authorization, project CRUD, upload session creation/finalization, plan endpoints, job status and download URL issuance.
Planner service	Builds compact model context, calls configured LLM provider, validates structured JSON with Zod, normalizes values and persists a versioned EditPlan.
Analysis worker	ffprobe, proxy and WAV generation, transcription, silence/scene/beat detection; persists analysis metadata.
Render worker	Loads validated plan by IDs, compiles deterministic FFmpeg arguments, renders, uploads output, reports progress and completion.
MongoDB	System of record for users, projects, assets, analysis results, edit plans, timelines, versions and render jobs.
S3	Immutable raw uploads, proxies, generated subtitle files, previews and final renders.
Redis / BullMQ	Durable asynchronous queues, job state/progress pub-sub, retry scheduling.

1.2 Key architectural rules

• The LLM returns data (Edit Plan JSON), never executable commands. Only deterministic server code compiles plans into FFmpeg argument arrays.
• The original source file is immutable. Every accepted edit creates a new plan/timeline version.
• Preview uses the 480p proxy; final export uses the original asset. Both use the same compiler.
• Workers receive identifiers, not large plan bodies, and load the latest validated data from MongoDB.
• Every database query is scoped by both userId and the relevant projectId where applicable.

2. API conventions

Base URL: https://<api-host>/api
Content type: application/json except multipart bytes sent directly to S3.
Authentication: Authorization: Bearer <access-token> (or secure session cookie).

Identifiers: public prefixed IDs such as prj_..., ast_..., and job_...; internal MongoDB ObjectIds are never exposed as authorization tokens.

2.1 Standard response and error envelope

```
{
  "success": true,
  "data": { "...": "..." },
  "requestId": "req_01J..."
}

{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "The request could not be accepted.",
    "details": [{ "field": "prompt", "issue": "Required" }]
  },
  "requestId": "req_01J..."
}
```

Use HTTP status codes consistently: 200 successful read/update; 201 resource created; 202 asynchronous job accepted; 400 malformed request; 401 unauthenticated; 403 unauthorized; 404 not found in the caller's tenant; 409 version/state conflict; 413 upload too large; 415 unsupported media; 422 semantic validation failure; 429 rate/quota limit; 500 unexpected server error; 503 dependency unavailable.

3. Internal API endpoint contracts

3.1 Endpoint catalogue

Method & route	Purpose	Typical response
POST /uploads	Create multipart upload session and presigned part URLs.	201 uploadId, assetId, partSize, part URLs, expiry.
POST /uploads/:id/complete	Verify uploaded parts, complete S3 multipart upload, create Asset and enqueue analysis.	202 assetId, analysisJobId, state=queued.
GET /projects/:id	Return project, asset metadata and compact analysis summary.	200 project object.
POST /projects/:id/prompt	Submit natural-language edit instruction; create plan version.	200/202 plan version and optional preview job.
GET /projects/:id/plans/:v	Fetch an immutable Edit Plan version.	200 plan envelope.
PATCH /projects/:id/plans/:v	Apply validated manual patch and create a new version.	200 new version; 409 on stale base version.
POST /projects/:id/preview	Enqueue 480p preview for a plan version.	202 render job.
POST /projects/:id/export	Enqueue final render for a plan version/profile.	202 render job.
GET /jobs/:id	Read current job state and progress.	200 job status.
GET /jobs/:id/events	Subscribe to progress/state/error/done events using SSE.	text/event-stream.
GET /jobs/:id/download	Issue short-lived presigned download URL when job is done.	200 URL and expiry.

3.2 Chunked / multipart video upload

The browser requests an upload session from Express, then transfers file parts directly to S3 using presigned URLs. Express does not proxy the video stream. The client records each part number and ETag and submits them when finalizing.

```
POST /api/uploads
Authorization: Bearer <token>
Content-Type: application/json

{
  "projectId": "prj_8f2a91c4",
  "fileName": "source-video.mp4",
  "contentType": "video/mp4",
  "sizeBytes": 734003200,
  "checksumSha256": "<optional-base64-or-hex>"
}

201 Created
{
  "success": true,
  "data": {
    "uploadId": "upl_01J...",
    "assetId": "ast_01J...",
    "partSizeBytes": 16777216,
    "parts": [
      { "partNumber": 1, "url": "https://s3-presigned-url...", "expiresAt": "..." },
      { "partNumber": 2, "url": "https://s3-presigned-url...", "expiresAt": "..." }
    ],
    "expiresAt": "..."
  }
}

POST /api/uploads/upl_01J.../complete
{
  "parts": [
    { "partNumber": 1, "etag": "\"etag-part-1\"" },
    { "partNumber": 2, "etag": "\"etag-part-2\"" }
  ]
}
```

```
202 Accepted
{
  "success": true,
  "data": {
    "assetId": "ast_01J...",
    "analysisJobId": "job_01J...",
    "state": "queued"
  }
}
```

Upload requirements: authorize project ownership before issuing URLs; enforce maximum file size and allowed content types; validate actual media with ffprobe after upload; use S3 multipart abort/expiry cleanup for abandoned sessions; presigned upload URLs expire after 15 minutes; never trust client-supplied MIME type or filename as proof of file content.

3.3 Prompt submission and plan generation

```
POST /api/projects/prj_8f2a91c4/prompt
{
  "prompt": "Make it cinematic, add animated subtitles and smooth transitions.",
  "baseVersion": 3,
  "requestPreview": true
}

200 OK
{
  "success": true,
  "data": {
    "planId": "pln_01J...",
    "version": 4,
    "parentVersion": 3,
    "state": "planned",
    "plan": {
      "schemaVersion": "1.0",
      "projectId": "prj_8f2a91c4",
      "baseAssetId": "ast_01",
      "version": 4,
      "parentVersion": 3,
      "global": { "aspect": "16:9", "fps": 30, "resolution": "1080p" },
      "layers": [],
      "meta": { "prompt": "...", "model": "provider-model-id" }
    },
    "previewJobId": "job_01J..."
  }
}
```

The server builds model context from the user prompt, current plan (if present), compact analysis (transcript/word timings, silences, scene cuts, beat grid, duration, resolution and fps), and approved catalog entries. The raw video is not sent to the LLM. Follow-up prompts should use RFC 6902-style patches so unmentioned edits are preserved.

3.4 Manual plan edits, preview, export and download

```
PATCH /api/projects/prj_8f2a91c4/plans/4
{
  "baseVersion": 4,
  "patch": [
    { "op": "replace", "path": "/layers/lyr_s1/params/size", "value": 72 }
  ]
}

POST /api/projects/prj_8f2a91c4/preview
{ "planVersion": 5 }

POST /api/projects/prj_8f2a91c4/export
{ "planVersion": 5, "profile": "final_1080p" }

202 Accepted
{
  "success": true,
  "data": {
    "jobId": "job_01J...",
    "type": "render",
    "state": "queued",
    "eventsUrl": "/api/jobs/job_01J.../events"
  }
}

GET /api/jobs/job_01J.../download
```

```
200 OK
{ "url": "https://s3-presigned-get...", "expiresInSeconds": 3600 }
```

Use optimistic concurrency: the client sends baseVersion (and, for timeline writes, the current rev). If the server's current version differs, return 409 VERSION_CONFLICT with the latest version metadata; the client refetches before retrying. Do not silently overwrite another edit.

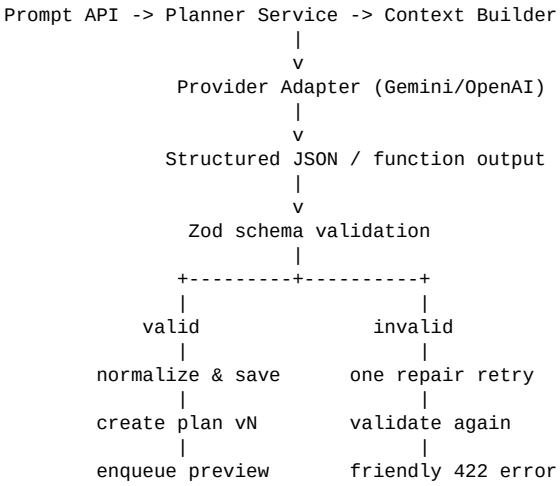
4. Third-party AI provider architecture

4.1 Provider adapter pattern

Keep provider-specific SDK calls behind a PlannerProvider interface. The planner service owns prompt construction, schema selection, validation, retries and persistence; provider adapters only translate the request and response format.

```
interface PlannerProvider {
  generateEditPlan(input: {
    systemInstructions: string;
    userPrompt: string;
    currentPlan?: EditPlan;
    analysisSummary: AnalysisSummary;
    catalog: AllowedCatalog;
    outputSchema: JSONSchema;
    maxOutputTokens: number;
    timeoutMs: number;
  }): Promise<{
    provider: "gemini" | "openai";
    model: string;
    structuredOutput: unknown;
    usage?: { inputTokens?: number; outputTokens?: number };
    providerRequestId?: string;
  }>;
}
```

Provider flow



4.2 Safe credential management

• Store provider keys only in server-side environment secrets or a managed secret store. Never ship keys to React, browser bundles, mobile clients, logs, or MongoDB documents.
• Use separate credentials for development, staging and production; rotate keys and restrict provider project permissions where supported.
• Keep provider selection and model name in server configuration. Persist only non-secret metadata such as provider, model identifier, usage counters and request ID in the Edit Plan metadata or job logs.
• Redact Authorization headers, API keys, user private data and sensitive prompt context from logs. Limit access to prompt/analysis logs.

4.3 Token, context and cost controls

The supplied specification does not define fixed token quotas or provider-specific model limits. Therefore, token budgets must be configurable per provider/model and enforced by the planner rather than hard-coded to one vendor.

Recommended controls:
1. Build compact context: send only the current plan, relevant transcript excerpts, scene/silence/beat summaries and allowed catalog metadata. Do not send raw video.
2. Set configurable max input/output token budgets and reject or summarize oversized context before provider

invocation.
3. Enforce per-user rate limits and per-project token/cost budgets; return 429 QUOTA_EXCEEDED when exhausted.
4. Use temperature 0.2 and structured output/function calling with the Edit Plan JSON schema.
5. Cap the plan at 200 layers and text at 120 characters; clamp numeric values to allowed ranges; reject unknown layer types and strip unknown fields.
6. Never allow the model to invent asset IDs or catalog references. Resolve every reference against server-side records.
7. Transcript text and user content are untrusted data, not system instructions. Keep them clearly separated from trusted system instructions.

4.4 Timeouts, retries and provider failure policy

Use a bounded provider timeout and a small retry policy for transient rate-limit/network/server failures. Apply exponential backoff with jitter and honor Retry-After when supplied. Do not blindly retry malformed output or authentication/permission errors. The existing functional specification calls for one automatic repair retry when schema validation fails; after that, return a friendly validation error.

Persist a planning job state such as queued, planning, planned or failed. If plan generation is moved fully asynchronous, return 202 with a jobId and expose its state through the same job status/SSE mechanism. Do not hold an HTTP request open while waiting through long provider delays.

4.5 Model output safety boundary

The LLM produces an Edit Plan data structure only. The server validates it with a Zod discriminated union, normalizes time ranges and parameter bounds, verifies catalog references, and saves a new immutable version. The FFmpeg compiler accepts only validated server-side plan data and builds an argument array; it must never interpolate model output or user text into a shell command.

5. Asynchronous processing and webhook specifications

5.1 Queue and worker model

Use separate BullMQ queues so analysis, preview and final rendering can scale independently. Queue payloads contain identifiers only; workers fetch validated plan/asset data from MongoDB and object storage.

Queue	Concurrency target	Timeout	Retry policy
analysis	2 per node	20 minutes	2 retries, exponential backoff
preview	3 per node	5 minutes	1 retry
render	1 per CPU group (GPU optional)	60 minutes	1 retry; OOM/timeout may be retried on larger worker

5.2 Worker lifecycle

```
API creates renderJobs record (state=queued)
        |
        v
BullMQ queue: { projectId, planVersion, jobId, type }
        |
        v
Worker claims job -> state=processing
        |
        +--> load asset / plan from MongoDB
        +--> compile deterministic FFmpeg args
        +--> run FFmpeg in isolated temp directory
        +--> parse -progress pipe:1
        +--> upload output to S3
        |
        +--> publish progress to Redis channel job:{id}
        +--> POST signed completion event to internal API
        |
        v
API updates renderJobs + project status
        |
        v
React receives SSE: progress / state / error / done
```

5.3 Internal worker completion callback (proposed)

The existing blueprint specifies that the worker reports progress through Redis and that the API relays progress over SSE. It also describes an S3 upload-completion webhook for ingest. The following callback contract formalizes render-worker completion as a proposed internal extension, not an existing provider requirement.

```
POST /api/internal/v1/worker-events
Content-Type: application/json
X-Worker-Id: render-worker-01
X-Event-Id: evt_01J...
X-Event-Timestamp: 1793822400
X-Event-Signature: sha256=<HMAC signature>

{
  "eventId": "evt_01J...",
  "eventType": "render.completed",
  "occurredAt": "2026-11-04T12:00:00Z",
  "jobId": "job_01J...",
  "projectId": "prj_8f2a91c4",
  "planVersion": 5,
  "profile": "final_1080p",
  "result": {
    "outputKey": "users/u1/projects/p1/renders/job_01J....mp4",
    "durationMs": 58400,
    "width": 1920,
    "height": 1080,
    "sizeBytes": 48200123
  }
}

200 OK
{
  "success": true,
```

```
  "data": { "accepted": true, "duplicate": false }
}
```

5.4 Event types and payloads

```
render.progress
{
  "eventId": "evt_...",
  "eventType": "render.progress",
  "jobId": "job_...",
  "projectId": "prj_...",
  "planVersion": 5,
  "progress": { "percent": 42, "outTimeMs": 24528000 }
}

render.failed
{
  "eventId": "evt_...",
  "eventType": "render.failed",
  "jobId": "job_...",
  "projectId": "prj_...",
  "error": {
    "code": "FILTER_ERROR",
    "message": "The requested effect could not be rendered.",
    "retryable": false
  }
}

analysis.completed
{
  "eventId": "evt_...",
  "eventType": "analysis.completed",
  "jobId": "job_...",
  "projectId": "prj_...",
  "assetId": "ast_...",
  "analysisResultId": "anl_..."
}
```

5.5 Signature verification and replay protection

• Sign the exact raw request body with HMAC-SHA256 using a dedicated worker-to-API secret. Compare signatures in constant time.
• Include event timestamp and reject requests outside a short configurable clock-skew window (for example, 5 minutes).
• Require a unique eventId and store processed event IDs with a unique index; duplicate delivery must return 200 without repeating side effects.
• Restrict the internal endpoint to worker network identities/private networking where possible. Rotate callback secrets independently from LLM provider keys.
• Validate that jobId, projectId, planVersion and outputKey match the stored renderJobs record and expected S3 prefix. Never trust a callback to set arbitrary output paths or another user's project status.

5.6 Idempotency and state transitions

The worker may retry callback delivery after a timeout. The API must make callback processing idempotent: first delivery updates the job; later deliveries with the same eventId are acknowledged as duplicates. State transitions should be monotonic and validated (queued → processing → done/failed). A late progress event must not move a completed job back to processing.

5.7 Browser progress: SSE contract

```
GET /api/jobs/job_01J.../events
Accept: text/event-stream
Authorization: Bearer <token>

event: progress
data: {"jobId":"job_01J...","state":"processing","percent":42}

event: state
data: {"jobId":"job_01J...","state":"uploading_output"}

event: done
data: {"jobId":"job_01J...","state":"done","downloadUrl":null}
```

```
event: error
data: {"jobId":"job_01J...","state":"failed","code":"FILTER_ERROR"}
```

SSE is the client-facing live update channel; the client may also call GET /jobs/:id after reconnecting to recover the latest durable state. The API relays Redis channel job:{id} messages to authorized subscribers. Send heartbeat comments to keep connections alive, close streams when the job is terminal, and verify project ownership before opening a stream.

6. Data ownership, status and storage contracts

6.1 Relevant MongoDB collections

Collection	API / worker responsibility
projects	Project metadata, userId, status, settings, baseAssetId, timelineId, headVersion and lightweight version index.
assets	Uploaded media metadata, S3 keys, duration, dimensions, codec, checksum and processing state.
analysisResults	Transcript, silence ranges, scene cuts, beat grid and audio analysis; one result per source asset.
editPlans	Immutable prompt + validated JSON plan, version, parentVersion, provider/model metadata.
timelines / timelineVersions	Resolved editable output-time timeline and immutable snapshots; optimistic rev counter.
renderJobs	Job type, projectId, planVersion, profile, state, percent, outputKey, error and timings.
chatMessages	Prompt and assistant response history, paginated separately from project/timeline documents.

6.2 Object storage key convention

```
s3://<bucket>/
  users/{uid}/projects/{pid}/
    raw/{assetId}.mp4              # original upload, immutable
    proxy/{assetId}_480p.mp4       # analysis and preview source
    audio/{assetId}.wav            # 16 kHz mono transcription input
    subs/{planVersion}.ass         # generated subtitle file
    previews/{planVersion}.mp4     # 480p preview
    renders/{jobId}.mp4            # final export
  library/music/
  library/luts/
  library/templates/
  library/fonts/
```

6.3 Job state model

Upload: created → uploading → completed / expired / failed.
Analysis: queued → analyzing → ready / failed.
Planning: planning → planned / failed.
Preview/render: queued → processing → uploading_output → done / failed / canceled.

A failed render must not corrupt or delete the validated plan. The user can correct the prompt and create another plan version. A completed render stores the output object key; the API issues a time-limited download URL only after checking ownership.

7. Security, resilience and performance

7.1 Security checklist

• Authentication and tenant scoping on every project, asset, plan, job, SSE and download operation.
• Presigned S3 upload URLs expire in 15 minutes; download URLs expire in 1 hour. Never persist or log the full signed URL.
• Enforce upload size limits, MIME sniffing and ffprobe validation before processing.
• Do not interpolate user text or LLM output into shell strings. Invoke FFmpeg with child_process.spawn('ffmpeg', argsArray), not a shell command string.
• Escape text written into ASS subtitle files; resolve file paths and catalog assets by server-side IDs only.
• Rate-limit prompt submissions and cap token usage per project/user.
• Workers run stateless in isolated containers with CPU/memory limits, per-job temporary directories and cleanup in finally blocks. Worker network access is limited to required storage/services.
• Use licensed catalog assets for music, LUTs, templates and fonts.

7.2 Retry and error classification

Error code	Policy
INPUT_CORRUPT	No retry; mark asset/job failed and request a new upload.
FILTER_ERROR	No automatic retry; log plan version and sanitized FFmpeg args for diagnosis.
OOM_OR_TIMEOUT	Retry once on a larger worker class.
S3_TRANSIENT	Retry with exponential backoff.
PROVIDER_RATE_LIMIT / PROVIDER_5XX	Bounded retry with backoff and Retry-After; do not exceed request deadline.
PROVIDER_AUTH_ERROR	No retry; alert operators and return a dependency error.
PLAN_VALIDATION_ERROR	One repair retry; then return a clear validation response.

7.3 Performance targets from the functional specification

- Plan generation: under 15 seconds at p95.
- Analysis: under 0.5× real time for a 10-minute video.
- Preview render: under 0.5× real time at 480p.
- Scrubbing latency: under 100 ms using the 480p proxy.
- Timeline canvas draw: under 4 ms per frame at 1080p CSS size.

7.4 Operational observability

Attach requestId, userId, projectId, assetId, planVersion and jobId to structured logs where available. Record queue wait time, provider latency/token usage, validation failures, FFmpeg duration, upload duration, retry count and terminal state. Redact credentials, signed URLs and unnecessary user content. Alert on queue backlog, repeated provider failures, worker crashes and jobs exceeding expected duration.

8. End-to-end sequence examples

8.1 Upload and analysis

```
1. React -> POST /uploads
2. API -> create upload session; return presigned S3 part URLs
3. React -> PUT file parts directly to S3
4. React -> POST /uploads/:id/complete with part ETags
5. API -> complete multipart upload; create Asset; enqueue analysis job
6. Analysis worker -> ffprobe, proxy, WAV, transcript, silence/scene/beat detection
7. Worker -> save analysisResults; update project/asset state
8. React -> GET project / subscribe to job SSE for current state
```

8.2 Prompt, preview and final export

```
1. React -> POST /projects/:id/prompt with prompt + baseVersion
2. API/Planner -> build compact context; call Gemini/OpenAI adapter
3. Planner -> validate with Zod; normalize; persist editPlans version N
4. API -> enqueue preview job and return jobId
5. Render worker -> compile validated plan; render 480p; upload preview
6. Worker -> Redis progress; signed completion callback to API
7. API -> update renderJobs; SSE emits progress/done
8. React -> preview playback; user approves or submits another prompt
9. React -> POST /projects/:id/export with planVersion + profile
10. Render worker -> render from immutable raw source; upload final MP4
11. API -> mark job done; GET /jobs/:id/download returns signed URL
```

9. Implementation checklist

Phase 1 — API foundation

■ Express app, auth middleware, request IDs and standard error handler.
■ Project ownership middleware and MongoDB models/indexes.
■ Upload session and multipart completion endpoints.
■ S3 presigned URL generation and cleanup policy.

Phase 2 — AI planner

■ Provider adapter interface and secret configuration.
■ Context builder and Edit Plan JSON schema.
■ Zod validation, normalization, catalog reference checks and repair retry.
■ Versioned plan persistence and prompt rate/token quotas.

Phase 3 — Background processing

■ BullMQ queues and worker concurrency/timeouts.
■ Analysis worker and render worker.
■ Redis progress publishing and SSE relay.
■ Signed, idempotent worker completion callback.

Phase 4 — Reliability and security

■ Tenant isolation tests for every endpoint.
■ Upload size/MIME/ffprobe tests.
■ Duplicate callback and out-of-order event tests.
■ Provider timeout, rate-limit and invalid-JSON tests.
■ Retry, cleanup, observability and load tests.

10. Source alignment and open decisions

This design is aligned with the two companion blueprints: the functional specification's architecture, API surface, S3 lifecycle, BullMQ queues, Edit Plan validation, FFmpeg pipeline and SSE progress; and the database blueprint's projects, assets, analysisResults, editPlans, renderJobs, versioning and tenant ownership model.

Items not fixed by the companion documents and therefore left configurable or proposed here: exact provider model IDs and token quotas; upload maximum size and chunk size; exact API authentication mechanism; internal callback HMAC secret

provisioning; callback retry schedule; and whether prompt generation itself is synchronous or represented as a planning job. Confirm these during implementation and record final values in environment/configuration documentation.

End of document